



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING

A
MINOR PROJECT PROPOSAL
ON
**CUSTOMER SEGMENTATION AND INTELLIGENT MOBILE
RECOMMENDATION SYSTEM**

SUBMITTED BY:

AAYUSH CHHUKA (NCE080BCT002)
HOM RAJ BHANDARI (NCE080BCT016)
SUDIP KUMAR TAMANG (NCE080BCT042)
SURESH KHADKA (NCE080BCT046)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

LALITPUR, NEPAL

FEBRUARY, 2026

CONTENTS

List of Figures	iii
List of Tables	iv
List of Abbreviations	v
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope	2
2 Literature Review	4
2.1 Related Work	4
2.2 Related Theory	5
2.2.1 Gradient Boosting Machines	5
2.2.2 XGBoost Overview	5
2.2.3 Mathematical Foundation	5
2.2.4 Key Technical Features	7
2.2.5 Interpretability Considerations	7
2.2.6 Why XGBoost for This Project	8
2.3 Website Development for End Users	8
2.3.1 User Interface Design	8
2.3.2 Retailer Functionality	8
2.3.3 Customer Functionality	9
2.3.4 Recommendation Display	9
3 Proposed Methodology	10
3.1 Feasibility Study	10
3.1.1 Technical Feasibility	10
3.1.2 Economic Feasibility	11

3.1.3	Operational Feasibility	11
3.1.4	Legal and Ethical Feasibility	12
3.2	Requirement Analysis	12
3.2.1	Functional Requirements	12
3.2.2	Non Functional Requirements	13
3.3	Proposed System Design	13
3.3.1	System Architecture	13
3.3.2	System Flowchart	14
3.3.3	Data Preparation and Machine Learning Workflow	16
3.4	Implementation Details	18
3.4.1	Frontend Development	18
3.4.2	Backend Development	18
3.4.3	Machine Learning Model Integration	19
3.4.4	Database Schema	19
3.4.5	Testing Strategy	20
4	Time Schedule	22
5	Expected Output	23
5.1	Core Application Features	23
5.2	Technical Deliverables	24
5.3	Impact and Benefits	25
5.4	Impact and Benefits	25
	References	26

LIST OF FIGURES

I	System Flowchart Showing Retailer and Customer Workflows	14
II	Data Collection and Integration Process	16
III	Machine Learning Model Workflow	17
I	Gantt chart showing project timeline for Customer Segmentation and Intelligent Mobile Recommendation System	22

LIST OF TABLES

I Economic Feasibility Estimation 11

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
BCT	Bachelor in Computer Engineering
CPU	Central Processing Unit
CSS	Cascading Style Sheet
CSV	Comma Separated Values
GBM	Gradient Boosting Machine
HTML	HyperText Markup Language
ID	Identifier
ML	Machine Learning
MP	Megapixel
UX	User Experience
XGBoost	Extreme Gradient Boosting

1. INTRODUCTION

1.1 Background

Customer segmentation is the process of grouping buyers based on shared traits to better understand their needs. While traditional methods rely on basic details like age or income, these often miss the deeper preferences of modern mobile phone users. A customer's choice is now driven more by their "tech tier" (performance needs) and brand loyalty than by simple demographics.

Recent progress in machine learning (ML) allows us to find these hidden patterns automatically by looking at past sales and phone specifications. This project uses the XGBoost (Extreme Gradient Boosting) model, which is a powerful tool known for its high accuracy with organized data. Unlike older methods, it is excellent at handling complex categories, such as specific phone chipsets or locations, without needing manual data preparation [1].

Current market conditions reveal two main gaps in mobile retail. First, most recommendation systems need a lot of user history to work. This causes a "cold-start problem" where the system cannot help new users or suggest new products [2]. Second, there is a lack of simple tools to help marketing teams match products to the right audience. This often forces them to rely on slow and inaccurate guesswork.

This proposal presents an "integrated ML pipeline", a scalable solution built to modernise personalized retail. The platform allows retailers to automatically group customers by behavior, solve the cold-start problem through smart matching, and create a link between products and their ideal audiences.

1.2 Problem Statement

The mobile retail industry faces three major problems when trying to match the right phones with the right customers. First, retailers do not clearly understand what customers really want. They usually group people only by age or location, which does not show their real interests, so customers often receive advertisements for products they do not need. Second, recommendation systems need past purchase or browsing history to

give suggestions. This becomes a problem for new customers or newly launched phones because there is no previous data, so helpful recommendations cannot be provided. Third, even when retailers have customer data, they do not have a simple system to connect this data with marketing during a new product launch. Because of this, marketing teams depend on guesswork, which is slow and often not accurate.

1.3 Objectives

- I. To develop a brand recommendation system that suggests suitable mobile brands for customers based on purchasing trends in different locations.
- II. To classify customers into different technology preference groups, ranging from high-end users to budget buyers, based on the performance level of the phones they purchase.
- III. To analyze customer camera preferences by examining the camera specifications of previously purchased phones and identifying whether they prefer selfies, professional photography, or general use.
- IV. To design a system that matches newly launched phones with the most suitable customer groups, even if there is no previous sales data available.
- V. To identify brand-loyal and brand-switching customers by analyzing their purchase history over time.
- VI. To integrate all features into a simple and user-friendly web application that supports better marketing decisions and personalized product recommendations.

1.4 Scope

This project works in the mobile phone retail field and includes the following scope:

- I. It uses three real-world datasets including phone specifications, benchmark scores, and large-scale online purchase history data for analysis and modeling.
- II. It develops machine learning model called XGBoost to segment customers and recommend products based on location, behavior, technical features, and purchase patterns.

- III. It builds a Django backend system that can recommend products to users and also identify suitable customer groups for specific products.
- IV. It creates a web-based dashboard using HTML, CSS, and JavaScript to show customer segments and product recommendations in a simple way for marketing teams.

The limitations of this project are:

- I. It does not include real-time stock management or automatic price optimization.
- II. It does not include a mobile application for Android or iOS, and works only as a web system.
- III. It does not connect directly with any live e-commerce website or payment system.
- IV. It only uses the available open datasets and does not include additional private customer data.

2. LITERATURE REVIEW

2.1 Related Work

Chen and Guestrin [3] introduced XGBoost, a scalable tree boosting system that has become a benchmark for tabular data. Its regularized learning objective and efficient handling of mixed features make it ideal for customer segmentation tasks, directly supporting its use as the sole algorithm in this project.

Singh [4] developed a customer segmentation and recommendation system with a similar multi-dimensional approach, categorizing users into segments like Tech Savvy, Brand Lovers, and Camera Lovers based on purchase behavior. Their work, publicly available on GitHub, demonstrates a practical application of micro-segmentation for mobile phone recommendations using machine learning and a Flask backend. This project serves as a key reference for both the segmentation logic and the overall system architecture.

Qi et al. [5] proposed an XGBoost recommendation algorithm integrated with collaborative filtering to specifically address the cold-start problem. By using XGBoost to predict item preferences for new users, their work validates XGBoost’s capability to generate recommendations without historical purchase data, a core requirement for this project’s customer recommendation feature.

Ayyappan et al. [6] demonstrated a hybrid BiLSTM-XGBoost model for purchase intent prediction. While their work shows XGBoost’s effectiveness as a final classifier, this project focuses on a pure XGBoost approach for all segmentation and regression tasks, proving its standalone power for structured, tabular phone specifications and transaction data.

2.2 Related Theory

2.2.1 Gradient Boosting Machines

Gradient Boosting is a machine learning technique that builds a strong predictive model by combining many simple models in a sequential manner. The core idea is to train new models that correct the mistakes made by previously trained models. Each new model focuses on the difficult cases that earlier models could not handle correctly, gradually improving the overall prediction accuracy.

The process begins with a simple initial prediction, often just the average value of the target variable. In each subsequent step, a new model is trained on the errors or residuals of the combined previous models. These residuals represent the gap between the actual values and the predictions made so far. By learning to predict these residuals, each new model helps close this gap, pushing the ensemble toward better performance.

2.2.2 XGBoost Overview

XGBoost, which stands for Extreme Gradient Boosting, is an advanced implementation of gradient boosting that has become one of the most popular and successful machine learning algorithms. It was designed with both speed and accuracy in mind, making it suitable for a wide range of prediction tasks.

The algorithm builds an ensemble of decision trees one after another, where each tree learns from the mistakes of all previous trees. Unlike a random forest where trees are built independently, the trees in XGBoost work together as a team, with each new member focusing on what the team got wrong. This sequential learning approach allows the model to capture complex patterns in the data that simpler models might miss.

2.2.3 Mathematical Foundation

The learning process in XGBoost is guided by a carefully designed objective function that balances two competing goals. The first goal is to make accurate predictions, measured by a loss function that quantifies the difference between predicted and actual values. The second goal is to keep the model simple and general, measured by a regularization term that penalizes overly complex trees.

The overall objective that XGBoost minimizes during training can be expressed as:

$$\mathcal{L}(\theta) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (2.1)$$

where:

- $\mathcal{L}(\theta)$ is the overall objective function that XGBoost minimizes during training. It represents the total cost that balances prediction errors against model complexity.
- n is the total number of training samples in the dataset.
- y_i is the actual true value for the i -th training sample.
- $\hat{y}_i$ is the predicted value produced by the model for the i -th training sample.
- $l(y_i, \hat{y}_i)$ is the loss function that measures the difference between the actual value and the predicted value for a single sample. For regression tasks this is often squared error, and for classification tasks it is often logarithmic loss.
- K is the total number of decision trees in the ensemble.
- f_k represents the k -th decision tree in the ensemble.
- $\Omega(f_k)$ is the regularization term that penalizes the complexity of the k -th tree to prevent overfitting.

In this equation, the first term sums the loss across all n training samples, where l measures how wrong the prediction is for each sample. The loss function can be chosen based on the task, such as squared error for regression problems or logistic loss for classification. The second term adds up the complexity penalty for all K trees in the ensemble. This penalty term takes the form:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (2.2)$$

Here T represents the number of leaves in the tree, and w represents the scores assigned to each leaf. The gamma parameter controls how much the model is penalized for adding more leaves, helping to prevent the tree from growing too deep and memorizing noise in the data. The lambda parameter controls the magnitude of the leaf scores, encouraging the model to make more conservative predictions.

2.2.4 Key Technical Features

XGBoost introduces several innovations that contribute to its strong performance. One important feature is the use of second order derivatives during tree construction. While basic gradient boosting only uses the first derivative of the loss function, XGBoost also uses the second derivative, which provides more information about the shape of the loss surface. This allows the algorithm to make better decisions about how to split the data at each tree node.

Another crucial feature is the built in handling of missing values. In real world datasets, missing values are common and can be problematic for many algorithms. XGBoost learns the best direction to send missing values during training, effectively letting the data tell the algorithm how to handle gaps. This eliminates the need for separate imputation steps and often leads to better models.

Regularization is deeply integrated into XGBoost, going beyond simple pruning of trees after they are built. Both L1 and L2 regularization are applied directly to the leaf scores, which helps prevent individual trees from becoming too confident in their predictions. This regularization, combined with techniques like column subsampling where each tree only sees a random subset of features, helps the ensemble generalize well to new data.

2.2.5 Interpretability Considerations

While XGBoost models are more complex than single decision trees, they still offer ways to understand what the model has learned. Feature importance scores can be calculated to show which input variables had the biggest impact on predictions. For our camera preference model, this could reveal whether front camera resolution or rear camera resolution matters more for classifying customers into different camera preference groups.

The algorithm also provides tools for understanding how individual predictions are made. By examining the path a sample takes through each tree in the ensemble, we can see which features contributed to the final decision. This transparency is valuable when we need to explain why a particular phone was recommended to a customer or why a specific customer was assigned to a certain audience segment.

2.2.6 Why XGBoost for This Project

For this project, XGBoost handles the mixed data types present in our phone specifications and transaction records without requiring extensive preprocessing. The algorithm is robust to outliers in features like price and benchmark scores, which might otherwise skew the results. Its built in regularization helps ensure that our models will perform well on new customers and phones they have not seen during training.

The sequential learning nature of gradient boosting means XGBoost can focus on the difficult cases in our data. In customer segmentation, some customers may have clear preferences that are easy to predict, while others have more complex behavior patterns. XGBoost will naturally allocate more of its capacity to learning these challenging cases, potentially improving the quality of recommendations for all customers.

Finally, XGBoost has a proven track record in both academic research and industry applications involving tabular data. Its consistent performance across many different types of problems makes it a reliable choice for implementing all the models in our segmentation pipeline, from tech tier classification to brand loyalty prediction.

2.3 Website Development for End Users

2.3.1 User Interface Design

The website features a clean interface designed for two distinct user types: retailers and individual customers. The homepage presents clear navigation paths for both user groups, with a retail dashboard for business users and a customer portal for individual phone shoppers. The design prioritizes simplicity, ensuring that users can easily access the recommendation features without technical expertise.

2.3.2 Retailer Functionality

Retailers can access a dedicated dashboard by logging into their accounts. The main feature allows them to enter a location and receive instant category predictions for phones popular in that area. The system displays top mobile phones along with their specifications, camera tier classifications, and tech tier designations. This helps retailers understand which phones to stock based on local customer preferences and target their marketing efforts toward specific audience segments.

2.3.3 Customer Functionality

Individual customers can register as new users or log in with their existing customer ID. New users are guided through a simple preference setup process where they select their camera requirements, technical specifications preferences, and provide their geographical information. Returning users can quickly access personalized recommendations by entering their customer ID. The system then generates tailored phone suggestions that match each customer's unique preferences and location.

2.3.4 Recommendation Display

When recommendations are generated, the website presents them in an easy to scan format showing top matching phones with their key specifications. Each recommended phone includes its camera tier classification, tech tier designation, and a match score explaining why it suits the user's preferences. Customers can compare multiple options and explore detailed specifications to make informed purchasing decisions.

3. PROPOSED METHODOLOGY

3.1 Feasibility Study

The feasibility study evaluates whether the proposed customer segmentation and mobile recommendation system can be realistically developed and deployed under existing constraints.

The project flow follows standard SDLC phases from planning and analysis through implementation, testing, and deployment.

3.1.1 Technical Feasibility

The proposed system is technically feasible given the team's proficiency and access to essential tools and technologies.

Technology Stack:

- **Frontend:** React with HTML, CSS, and JavaScript for responsive website development
- **Backend:** Python with Django framework for server-side logic.
- **Machine Learning:** XGBoost library for all segmentation and recommendation models
- **Database:** MySQL for robust relational data management, handling user profiles, customer preferences, and phone specifications with efficient query performance
- **Version Control:** Git and GitHub for collaborative development and code management

Team Skills: The development team has foundational skills in Python programming, web development with Django, database management, and basic machine learning concepts. While XGBoost implementation may require additional learning, extensive documentation and community resources are available to support the team.

Development Tools: VS Code for code editing, Google Colab for all data science and model deployment, Git and GitHub for version control, Postman for API testing, and Django's built-in development server will be used throughout development.

3.1.2 Economic Feasibility

The proposed system is economically feasible as it relies entirely on open-source tools and free development platforms. The Django framework, Python programming language, XGBoost library, and MySQL database are all freely available with no licensing costs.

Cost assumptions are based on academic project requirements and free-tier boundaries of selected services.

Table I: Economic Feasibility Estimation

Item	Estimated Cost
Development Tools	Rs. 0 (open source)
Python Libraries	Rs. 0 (XGBoost, pandas, numpy, scikit-learn all open source)
Machine Learning Models	Rs. 0 (no licensing costs for model development)
Dataset Acquisition	Rs. 0 (Kaggle public datasets)
Domain Name	Rs. 0 (free for academic demonstration)
Web Hosting	Rs. 0 (local hosting or free tier cloud)
Miscellaneous	Rs. 500-1,000
Total Initial Cost	Rs. 500-1,000

For academic demonstration purposes, local hosting on development servers will suffice. Commercial deployment would require cloud hosting and domain registration expenses as the user base grows.

3.1.3 Operational Feasibility

The system is operationally feasible as it aligns well with the team's development capabilities and workflow. The web-based platform can be accessed through any modern browser on desktop and mobile devices, eliminating the need for platform-specific development. The interface is designed to be intuitive for both retailers seeking market insights and customers looking for personalized phone recommendations.

The platform requires minimal training for end users. Retailers can easily enter locations and view category predictions, while customers can register their preferences and receive recommendations with just a few clicks. The simple two-pathway design ensures high usability for both technical and non-technical users.

3.1.4 Legal and Ethical Feasibility

- I. All user data used in this project is sourced from publicly available datasets on Kaggle , which contain no personally identifiable information.
- II. The phone specifications dataset, Antutu benchmark scores, and eCommerce purchase transactions are all obtained from Kaggle’s public repositories [7] and [8] and web scraping from [9] for academic research and educational purposes.
- III. Customer purchase history and preferences used for training the XGBoost models are derived from anonymized Kaggle datasets with all identifying information removed.
- IV. The project follows ethical guidelines for machine learning projects including transparency in data sources, responsible use of public datasets, and prevention of any potential misuse.
- V. As an academic project, the system is not intended for commercial deployment and will not process real user data beyond controlled testing with synthetic information.
- VI. Location information in the datasets is used solely for geographic brand recommendation models and exists only at city level with no individual address details.
- VII. All machine learning models are trained exclusively on Kaggle’s publicly available datasets, ensuring compliance with data usage terms and avoiding any privacy concerns.
- VIII. The project acknowledges and follows Kaggle’s dataset terms of use, which permit academic and research applications of the provided data.
- IX. No proprietary or confidential information is used in model training, and all datasets are properly credited to their original sources on Kaggle.

3.2 Requirement Analysis

3.2.1 Functional Requirements

The system allows retailers and customers to register using email and password , and create profiles with basic details and location. Retailers can use a dashboard to enter a

location and get predictions of popular phones in that area, along with complete specifications, camera and tech category insights, and audience trends for better inventory planning. Customers can set preferences for camera quality, performance, and budget, or use customer ID to access personalized recommendations with match reasons and comparison options. The platform also classifies users by tech level, camera preference, regional brand popularity, and brand loyalty behavior. For newly launched phones, it predicts the most suitable audience by combining multiple model outputs.

3.2.2 Non Functional Requirements

The system is designed to be fast, easy to use, secure, reliable, and scalable. Model predictions are expected to return quickly, pages should load smoothly, and users should get search results without delay even when many people use the platform at the same time. The interface should be simple and clear for both technical and non technical users, with consistent design, easy navigation, and helpful error messages. The application should remain available throughout the day with stable uptime and proper handling of temporary backend failures. It should work across major browsers and different screen sizes, while also handling slow internet connections without crashes or session loss. The architecture should support future growth in users and data, allow new machine learning models to be added easily, and handle large scale batch predictions. The codebase should remain maintainable through modular design, coding standards, documentation, and independent model retraining. Reliability is supported by proper input validation, structured error responses, prediction confidence indicators, and detailed logging for monitoring and debugging.

3.3 Proposed System Design

3.3.1 System Architecture

Customer Segmentation and intelligent Mobile Recommendation System follows a modular three tier architecture with clear separation between data, model, and application layers. The frontend is developed using HTML, CSS, and JavaScript, providing a responsive web interface for both retailers and customers. The Django backend serves as the application layer, handling HTTP requests, user authentication, and orchestrating calls to the machine learning models. The model layer consists of trained XGBoost

models that perform the core segmentation and recommendation tasks. MySQL serves as the relational database for storing user profiles, customer preferences, phone specifications, and recommendation history. The architecture ensures that each component can be developed, tested, and maintained independently while working together seamlessly through well defined APIs.

3.3.2 System Flowchart

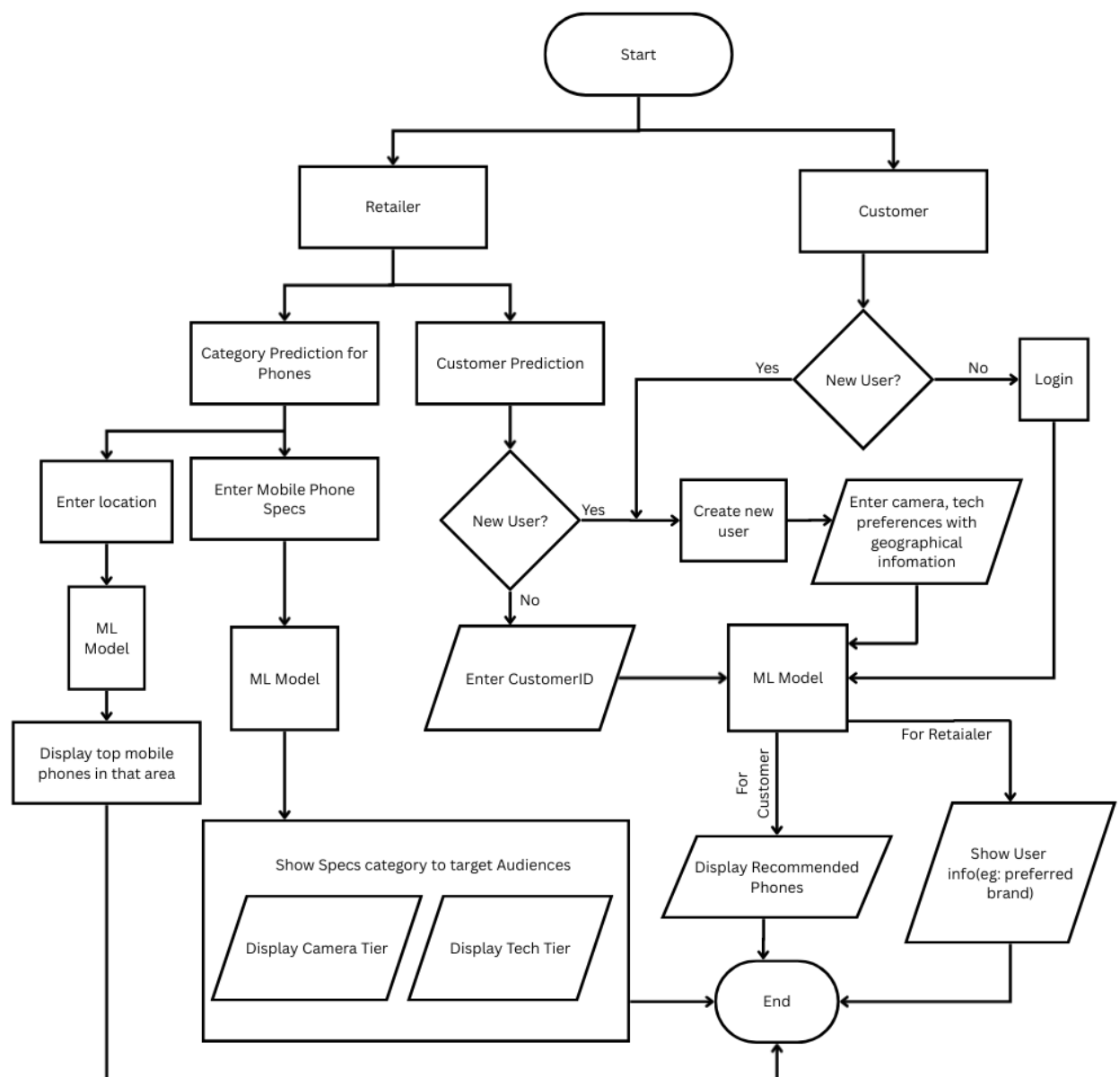


Figure I: System Flowchart Showing Retailer and Customer Workflows

The system flowchart illustrates the complete workflow for both retailer and customer interactions with the platform. The process begins at the start node and branches into two distinct pathways based on user type, ensuring each user group follows a tailored flow suited to their specific needs.

Retailer Pathway: Retailers access the category prediction feature by first entering a location of interest, such as a specific city or area where they operate or plan to launch marketing campaigns. They then provide mobile phone specifications including brand, processor, RAM, storage, and camera details. This input is processed by the machine learning model, which analyzes historical purchase data and geographic trends to generate predictions about phone popularity in that specific area. The system then displays the top mobile phones recommended for that location, along with specification categories that help retailers understand which audience segments to target. Additional outputs include camera tier classifications showing whether phones appeal to selfie focused users, photography enthusiasts, or balanced users, and tech tier designations indicating performance expectations from entry level to flagship enthusiasts. These comprehensive insights empower retailers to make data driven inventory decisions, optimize marketing campaigns for specific customer segments, and allocate stock strategically across different store locations based on local preferences.

Customer Pathway: Customers enter the customer prediction flow where the system first checks whether they are a new or existing user through a conditional branching node. New users are directed to a registration process where they create a comprehensive profile by entering their camera preferences indicating whether they prioritize selfies, professional photography, or balanced use, their tech expectations ranging from basic functionality to cutting edge performance, and their geographical information such as city or area of residence. Returning users can bypass registration by simply entering their unique customer ID to log in directly. Once authenticated, all customer inputs regardless of user type are processed by the machine learning model, which applies the trained XGBoost algorithms to analyze preferences against the phone specifications database. The system then generates personalized phone recommendations ranked by match scores. The display interface shows recommended phones along with detailed user information including preferred brands predicted by the geographic model, camera tiers matching their photography preferences, and tech tiers aligned with their performance

expectations. Additional context such as confidence scores for borderline predictions and explanations of why each phone was selected helps customers make informed decisions. The workflow concludes at the end node after successfully displaying all relevant outputs to both user types, completing the full system interaction cycle.

3.3.3 Data Preparation and Machine Learning Workflow

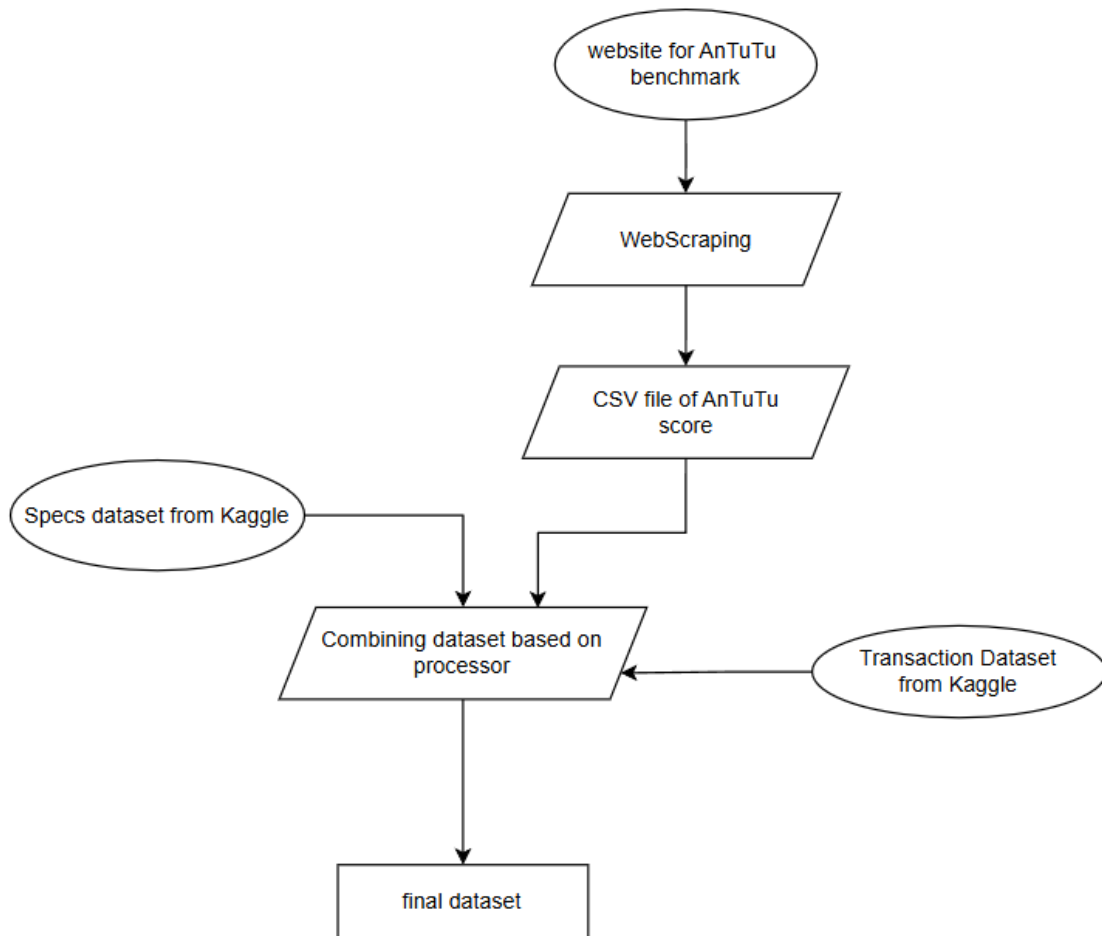


Figure II: Data Collection and Integration Process

The data collection process begins with three primary sources. Phone specifications are obtained from a Kaggle dataset, providing comprehensive hardware details including processor, RAM, storage, and camera information. AnTuTu benchmark scores are acquired through web scraping from official websites, resulting in a CSV file containing performance scores for various chipsets. Transaction data including customer purchase history is sourced from a Kaggle eCommerce dataset. These three datasets are then combined based on processor and phone model information to create a final integrated

dataset that links hardware specifications with performance scores and market transactions. This merged dataset serves as the foundation for all subsequent model training and analysis.

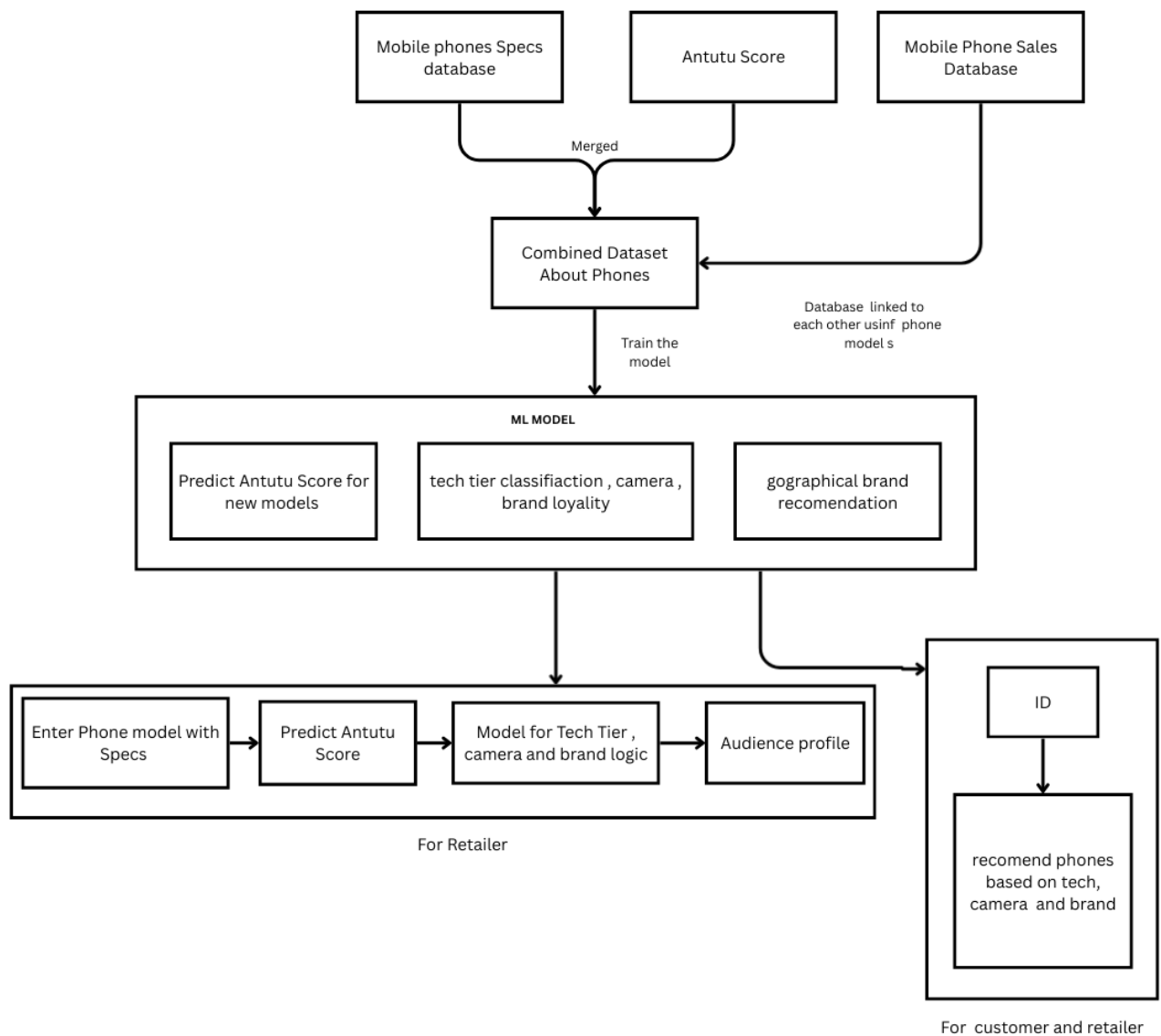


Figure III: Machine Learning Model Workflow

The machine learning workflow illustrates how the integrated dataset is used to train a unified XGBoost model. The mobile phone specifications database, Antutu scores, and mobile phone sales database are merged, creating a combined dataset about phones. This dataset is used to train the machine learning model, which learns to predict Antutu scores for new phone models based on their specifications. The model also performs tech tier classification, camera preference analysis, brand loyalty identification, and geographic

brand recommendation. When a retailer enters a phone model with specifications, the system predicts its Antutu score and applies tech tier, camera, and brand logic to generate a complete audience profile. For customer facing recommendations, the system takes customer ID as input and recommends phones based on matching tech tier, camera preferences, and brand affinity, serving both retailer and customer needs through an integrated pipeline.

3.4 Implementation Details

3.4.1 Frontend Development

The frontend will be developed using HTML, CSS, and JavaScript, providing a responsive web interface accessible on desktop and mobile browsers. The system includes authentication pages with login and registration forms for retailers and customers, allowing them to select their roles. The retailer dashboard provides a location input form, displays prediction results, shows audience insights through visualizations, and allows browsing of the phone catalog. The customer portal enables customers to log in using their ID, manage their preferences, and view personalized recommendations with match scores. The new user registration flow is a multi-step form that collects information about camera preferences, technical expectations, and location details. The recommendation display presents top phones in card format with specifications, match scores, camera tier badges, and tech tier labels. The phone details view shows complete specifications of selected phones, including predicted Antutu scores. The admin panel includes user management features, system monitoring tools, and model performance metrics.

The frontend will communicate with the Django backend. Bootstrap will be used for responsive design and consistent styling. JavaScript will handle dynamic content updates and form validation without page reloads.

3.4.2 Backend Development

The backend of the system is built using the Django web framework, which provides a strong and scalable foundation for the application. Django handles HTTP requests, URL routing, user authentication, and session management, and also offers built-in protection against common web security threats. The system uses a MySQL relational database

to store user profiles, customer preferences, phone specifications, and recommendation history, ensuring reliable data storage and efficient query performance. The authentication system is based on Django's built-in email and password mechanism, extended with phone-based OTP verification for better security. Additionally, a model integration layer is implemented using custom Django modules that load trained XGBoost models and provide prediction interfaces for machine learning models.

The backend follows Model View Template architecture with clear separation of concerns. Database models define schemas for users, phone specifications, and customer preferences. Views contain business logic for processing requests and invoking models. Templates render HTML pages with dynamic content.

3.4.3 Machine Learning Model Integration

The machine learning model is implemented using the XGBoost library and integrated into the Django backend:

The system uses a single XGBoost based machine learning model trained on a merged dataset combining mobile phone specifications, Antutu benchmark scores, and mobile phone sales data. The database links all information through phone models, enabling comprehensive analysis. When a retailer enters a phone model with specifications, the model predicts its Antutu score and then determines the appropriate tech tier classification, camera preference category, brand loyalty pattern, and geographic brand recommendation to generate a complete audience profile. For customer facing recommendations, the system takes customer ID as input and recommends phones based on matching tech tier, camera preferences, and brand affinity, serving both retailer and customer needs through an integrated pipeline.

All models are serialized using Python's pickle or joblib libraries and loaded into memory when the Django application starts. Model versioning is maintained to enable rollback if newer models underperform.

3.4.4 Database Schema

The MySQL database will implement core tables to store and manage all system data. These tables are expected to include user information for both retailers and customers, customer preference profiles capturing camera and tech expectations, phone specifications with complete hardware details, customer segmentation results, recommendation

history, purchase records for brand loyalty analysis, and aggregated location based brand statistics for geographic recommendations. The exact schema will be finalized during the system design phase based on detailed requirements analysis.

3.4.5 Testing Strategy

Unit Testing:

Unit testing focuses on verifying that individual components and functions work correctly in isolation. Each machine learning model will be tested independently to ensure it produces accurate predictions when given valid inputs. The Django backend will be tested to confirm they handle requests properly and return expected responses. Database queries and data validation functions will also be tested individually.

Integration Testing:

Integration testing examines how different parts of the system work together. This includes the connection between Django and the trained XGBoost models, which will be tested to ensure prediction requests are processed correctly and results are returned without errors. It also includes testing the communication between the frontend web interface and the Django backend to ensure data flows correctly. The integration between the backend and MySQL database will be verified to confirm that user profiles, preferences, and phone specifications are stored and retrieved accurately.

User Acceptance Testing:

User acceptance testing involves real users trying out the system to see if it meets their needs. Sample users representing retailers and customers will be asked to test the application and provide feedback. Retailers will test the location based prediction feature and audience insights. Customers will test the registration process, preference setup, and recommendation generation. Their feedback on usability, clarity of results, and overall experience will be collected and used to make improvements.

Performance Testing:

Performance testing checks how well the system responds under different conditions. Response times for location predictions and customer recommendations will be mea-

sured to ensure they stay under two seconds. The system will be tested with multiple users making requests at the same time to see if performance degrades. Database query speeds will be monitored as the number of phones and users grows. Testing will also cover how the application performs on different devices and browsers to ensure a consistent experience for all users.

4. TIME SCHEDULE

The Customer Segmentation and Intelligent Mobile Recommendation System project is scheduled according to the timeline illustrated in Figure I.

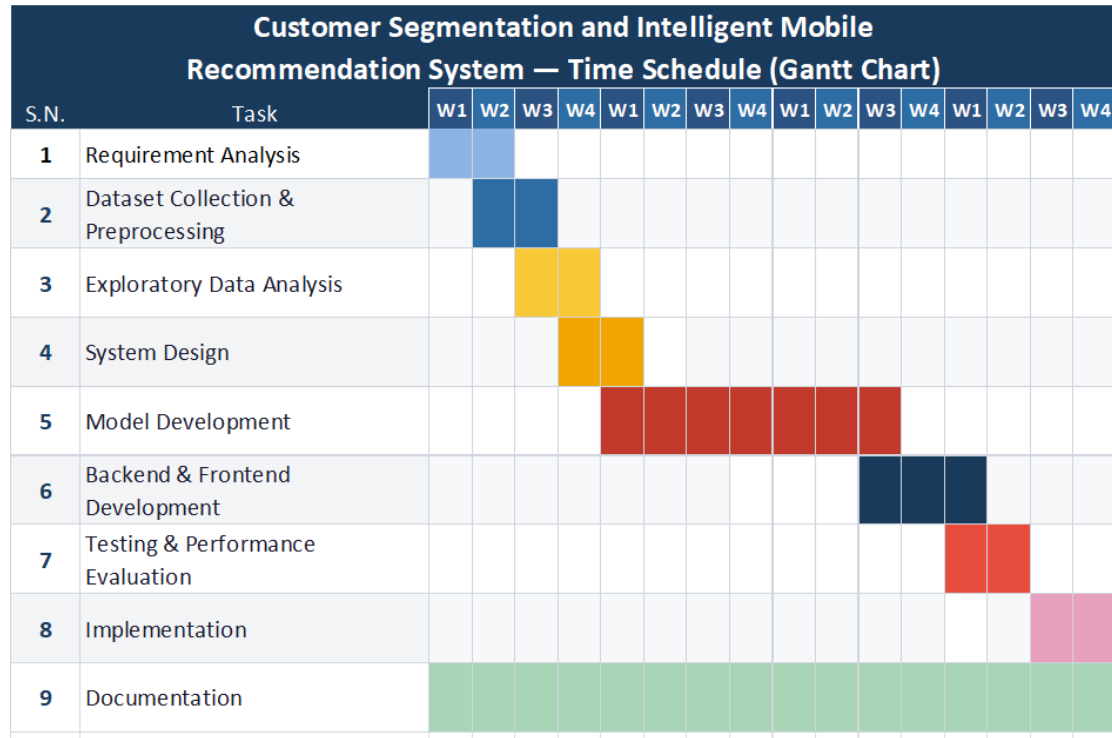


Figure I: Gantt chart showing project timeline for Customer Segmentation and Intelligent Mobile Recommendation System

The project begins with requirement analysis in the first week, followed by dataset collection and preprocessing continuing through week four. Exploratory data analysis is conducted from week three to week six, examining the three Kaggle datasets. System design takes place from week five to week eight, establishing the architecture for Django backend, MySQL database, and model integration.

Model development begins in week seven and continues through week eleven, covering the XGBoost model including geographic recommendation, tech tier classification, camera preference classification, score regression, brand loyalty classification, and the product to audience pipeline. Backend and frontend development runs from week nine through week twelve, building the Django backend and web interface. The documentation is done from beginning of the project to the end.

5. EXPECTED OUTPUT

5.1 Core Application Features

- **Web Application:**

Responsive web interface built with HTML, CSS, and JavaScript, providing dedicated dashboards for retailers and customers with clean and intuitive user experience.

- **Location Based Category Prediction:**

Retailers can enter any location to receive predictions about popular phone categories in that area, enabling data driven inventory decisions and targeted marketing campaigns.

- **Personalized Customer Recommendations:**

Customers receive tailored phone suggestions based on their camera preferences, tech expectations, and geographical location, with match scores explaining why each phone suits their needs.

- **Tech Tier Classification:**

Automatic segmentation of customers into five distinct tech tiers ranging from entry level users seeking basic functionality to tech enthusiasts wanting flagship performance, enabling precise matching with appropriate phones.

- **Camera Preference Classification:**

Customers are categorized into three camera segments including selfie focused users, photography enthusiasts, and balanced users based on front and rear camera specifications of phones they prefer.

- **Geographic Brand Intelligence:**

System analyzes purchase patterns across different cities to predict brand popularity by location, helping retailers understand regional preferences and customers discover locally popular brands.

- **Brand Loyalty Identification:**

Tracks customer purchase sequences to classify users into loyalty categories including loyal to a single brand, multi brand shoppers, or frequent switchers, enabling tailored retention strategies.

- **Product to Audience Pipeline:**

For new phones entering the market, the system predicts the ideal audience profile including expected tech tier, camera preference segment, and geographic brand affinity, helping marketing teams target the right customers before launch.

- **Weighted Recommendation Scoring:**

Recommendations combine tech tier match at 50 points, brand affinity at 30 points, and camera preference at 20 points to generate ranked results with clear explanations of why each phone was selected.

- **Audience Insights Dashboard:**

Retailers can view detailed segmentation data including estimated segment sizes, confidence scores for borderline predictions, and actionable insights for inventory planning and marketing campaigns.

5.2 Technical Deliverables

The project will deliver a fully functional web application built with HTML, CSS, and JavaScript, providing responsive interfaces for both retailers and customers. A Django backend will handle location prediction, customer recommendations, and product to audience analysis. The trained and validated XGBoost models will be delivered for geographic recommendation, tech tier classification, camera preference classification, score regression, brand loyalty classification, and the product to audience pipeline. A MySQL database with complete schema will store user profiles, customer preferences, phone specifications, and recommendation history. A model integration layer will ensure the model load correctly and deliver predictions within 500 milliseconds. Comprehensive documentation including API specifications, database schema, model performance reports, and the technical project report will also be provided.

5.3 Impact and Benefits

5.4 Impact and Benefits

For Retailers:

Retailers gain data driven inventory decisions through location based category predictions revealing local preferences. Audience insights including tech tiers, camera preferences, and brand segments enable targeted marketing campaigns. Understanding which customer segments to target reduces marketing waste, while regional brand intelligence enables smarter stock allocation across store locations.

For Customers:

Customers receive personalized recommendations matching their individual preferences rather than generic suggestions. Match scores and segment labels provide clear explanations building trust in the system. Time spent researching phones is reduced through curated suggestions aligned with their tech expectations and camera needs, leading to more satisfying purchase decisions.

For the Mobile Retail Ecosystem:

The project modernizes mobile phone retail through data driven segmentation and intelligent recommendations. It solves the cold start problem enabling recommendations for new phones and customers without historical data. The open source system using Python, Django, MySQL, and XGBoost can be extended to other product categories from electronics to automobiles, creating broader impact across retail sectors.

REFERENCES

- [1] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [2] Brij Kishore Pandey. The cold start problem in recommender systems, 2022. URL <https://www.freecodecamp.org/news/cold-start-problem-in-recommender-systems/>. Accessed: 2024-05-20.
- [3] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.
- [4] Bhavesh Singh. Customer segmentation recommendation. <https://github.com/bhaveshsingh0206/customer-segmentation-recommendation>, 2020. Accessed: 2026-02-26.
- [5] Peng Qi, Xiaolong Zhang, and Zhenyu Wang. An xgboost recommendation algorithm based on collaborative filtering. In *2018 IEEE 9th International Conference on Software Engineering and Service Science (ICSESS)*, pages 138–141, 2018. doi: 10.1109/ICSESS.2018.8663849.
- [6] Kavitha Ayyappan et al. Bidirectional lstm with xgboost for customer purchase intent prediction. *International Journal of Intelligent Systems and Applications in Engineering*, 11(4):284–293, 2023.
- [7] dilkushsingh. Smartphones dataset (august 2024). <https://www.kaggle.com/datasets/dilkushsingh/smartphones-dataset-upto-july24>, 2024. Accessed: 2026-02-27.
- [8] waqi786. Mobile sales dataset. <https://www.kaggle.com/datasets/waqi786/mobile-sales-dataset>, 2024. Accessed: 2026-02-27.
- [9] NanoReview. Phone list by antutu rating. <https://nanoreview.net/en/phone-list/antutu-rating>, 2026. Accessed: 2026-02-27.